الجامعة الأورومتوسطية بفاس
EUROMED UNIVERSITY OF FES
UNIVERSITÉ EUROMED DE FÈS

École d'Ingénierie Digitale et d'Intelligence Artificielle

AI Engineering

Multi-Agent Systems

Lab Report 2: ACL Communication in JADE

Academic Year: 2025-2026

Professor: Abderrahim Waga

Student Name: Esakhi Hajjar
Date: February 17, 2026

Contents

1 Introduction **3**

2 Environment Setup **3**
2.1 Tools Used . . . 3
2.2 Project Configuration . . . 3

3 Exercise 1: Unidirectional Communication **3**
3.1 Objective . . . 3
3.2 Implementation . . . 3
3.3 Complete Code Implementation . . . 3
3.3.1 AgentB.java . . . 3
3.3.2 AgentA.java . . . 4
3.4 Execution Command . . . 5
3.5 JADE GUI Interface . . . 5
3.6 Console Output . . . 6

4 Exercise 2: Bidirectional Communication **6**
4.1 Objective . . . 6
4.2 Implementation . . . 6
4.3 Complete Code Implementation . . . 6
4.3.1 AgentA_Ex2.java . . . 6
4.3.2 AgentB_Ex2.java . . . 7
4.4 Execution Command . . . 8
4.5 Console Output . . . 9

5 Exercise 3: Coordination of 3 Agents **9**
5.1 Objective . . . 9
5.2 Implementation . . . 9
5.3 Complete Code Implementation . . . 9
5.3.1 CoordinatorAgent.java . . . 9
5.3.2 Expert1.java . . . 10
5.3.3 Expert2.java . . . 11
5.4 Execution Command . . . 12
5.5 JADE GUI Interface . . . 12
5.6 Console Output . . . 13

6 Exercise 4: Decision Making and Negotiation **13**
6.1 Objective . . . 13
6.2 Implementation . . . 13
6.3 Complete Code Implementation . . . 13
6.3.1 BuyerAgent.java . . . 13
6.3.2 Seller1.java . . . 15
6.3.3 Seller2.java . . . 17
6.4 Execution Command . . . 18
6.5 JADE GUI Interface . . . 19
6.6 Console Output . . . 20

7 Errors Encountered and Solutions **20**
7.1 Port Already in Use 20

8 Conclusion **21**

1 Introduction

This lab focuses on implementing Agent Communication Language (ACL) in JADE (Java Agent DEvelopment Framework). The objectives include:

- Understanding ACL message exchange mechanisms
- Implementing unidirectional and bidirectional communication
- Coordinating exchanges between multiple agents
- Implementing decision-making and negotiation protocols

2 Environment Setup

2.1 Tools Used

- **IDE:** Eclipse
- **Framework:** JADE 4.6.0
- **Language:** Java
- **Package:** eu.uemf.agents

2.2 Project Configuration

The project was configured with the JADE library added to the build path. All agents were created in the package `eu.uemf.agents`.

3 Exercise 1: Unidirectional Communication

3.1 Objective

Implement one-way communication where Agent A sends an INFORM message to Agent B without expecting a reply.

3.2 Implementation

Agent B uses a `CyclicBehaviour` to continuously listen for incoming messages, while Agent A uses a `OneShotBehaviour` to send a single message.

3.3 Complete Code Implementation

3.3.1 AgentB.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;

public class AgentB extends Agent {

    @Override
    protected void setup() {
        System.out.println("Agent B is ready and listening...");

        // Add a cyclic behavior to continuously listen for messages
        addBehaviour(new CyclicBehaviour() {
            @Override
            public void action() {
                // Try to receive a message
                ACLMessage msg = receive();

                if (msg != null) {
                    // Message received
                    System.out.println("Agent B received: " + msg.getContent());
                    System.out.println("From: " + msg.getSender().getLocalName());
                } else {
                    // No message, block the behavior until one arrives
                    block();
                }
            }
        });
    }
}
```

3.3.2 AgentA.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.OneShotBehaviour;
import jade.lang.acl.ACLMessage;

public class AgentA extends Agent {

    @Override
    protected void setup() {
        System.out.println("Agent A is starting...");

        // Add a one-shot behavior to send a message once
        addBehaviour(new OneShotBehaviour() {
            @Override
            public void action() {
                // Create an INFORM message
                ACLMessage msg = new ACLMessage(ACLMessage.INFORM);

                // Set the receiver (Agent B)
```

```java
                msg.addReceiver(new AID("agentB", AID.ISLOCALNAME));

                // Set the message content
                msg.setContent("Hello B!");

                // Send the message
                send(msg);

                System.out.println("Agent A sent: Hello B!");
            }
        });
    }
}
```

3.4 Execution Command

Run Configuration → Arguments → Program arguments:

```
-gui agentB:eu.uemf.agents.AgentB;agentA:eu.uemf.agents.AgentA
```

3.5 JADE GUI Interface

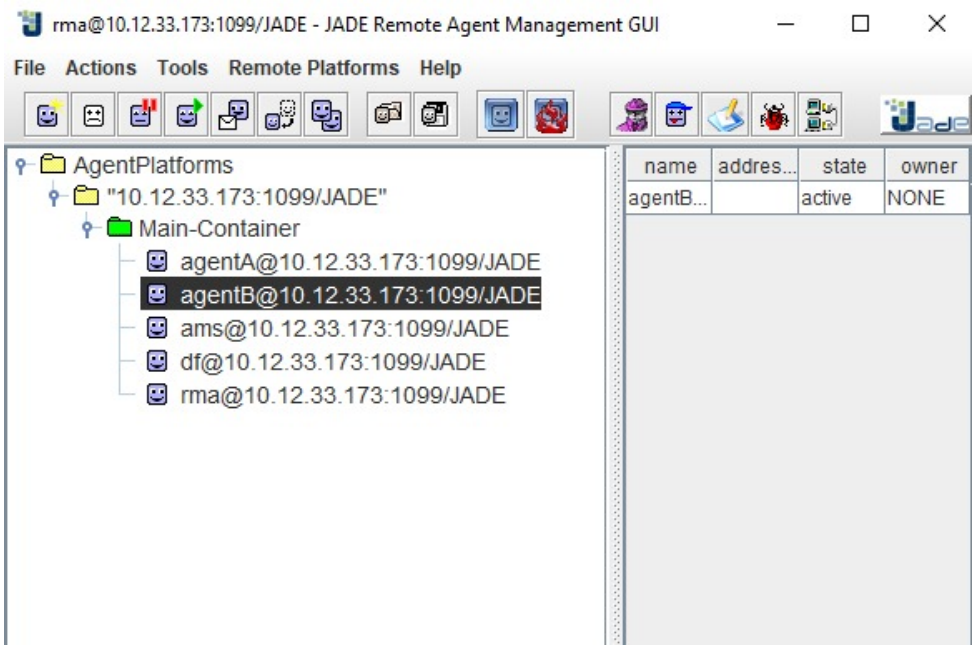

Figure 1: JADE GUI showing Agent A and Agent B

3.6 Console Output

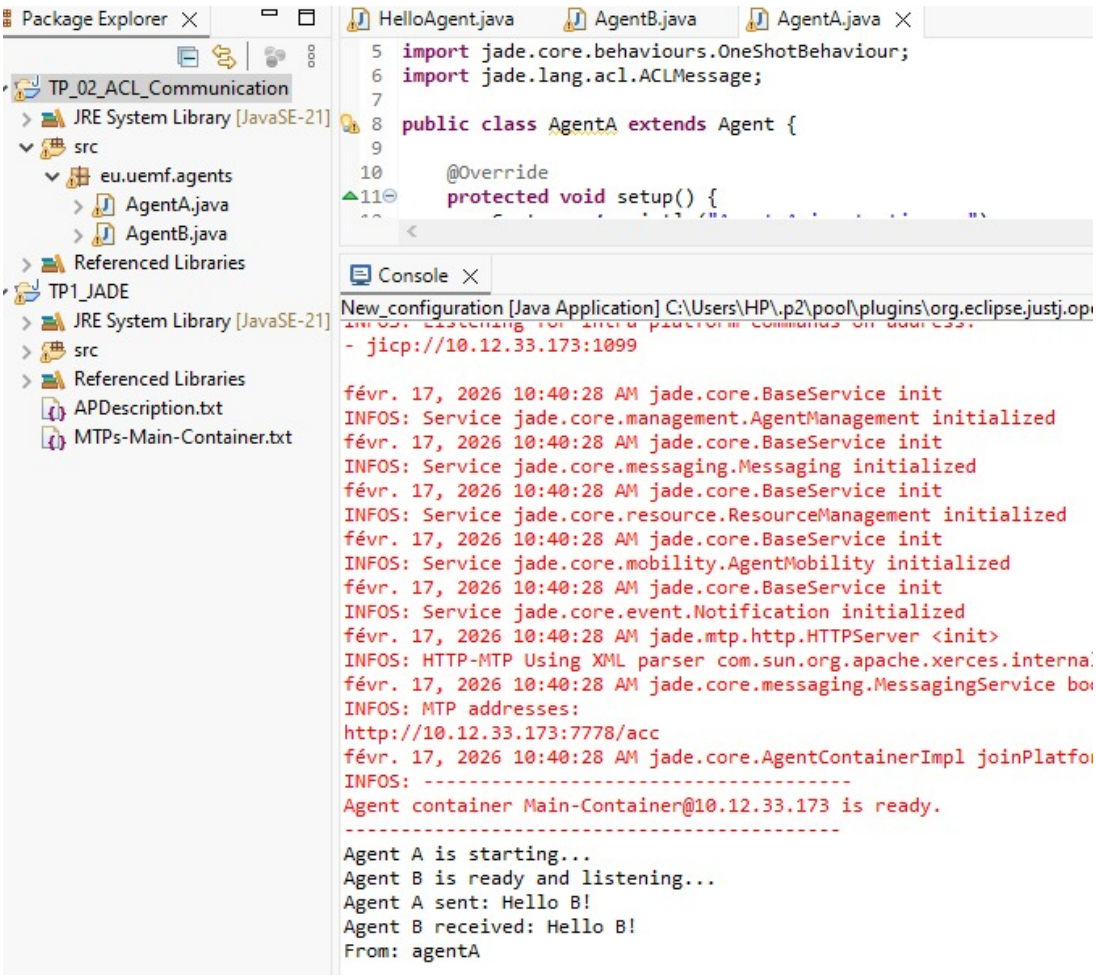

Figure 2: Console output showing message exchange

4 Exercise 2: Bidirectional Communication

4.1 Objective

Implement two-way communication where Agent A sends a REQUEST to Agent B, and Agent B responds with an INFORM message.

4.2 Implementation

Agent A sends a request and waits for a reply. Agent B processes the request using `createReply()` to automatically address the response to the correct sender.

4.3 Complete Code Implementation

4.3.1 AgentA_Ex2.java

```
package eu.uemf.agents;

```

```java
import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.Behaviour;
import jade.lang.acl.ACLMessage;

public class AgentA_Ex2 extends Agent {

    @Override
    protected void setup() {
        System.out.println("Agent A (Ex2) is starting...");

        addBehaviour(new Behaviour() {
            private boolean replyReceived = false;
            private int step = 0;

            @Override
            public void action() {
                switch (step) {
                    case 0:
                        // Send REQUEST message
                        ACLMessage request = new ACLMessage(ACLMessage.REQUEST);
                        request.addReceiver(new AID("agentB", AID.ISLOCALNAME));
                        request.setContent("What is the temperature?");
                        send(request);

                        System.out.println("Agent A sent REQUEST: What is the temperature?");
                        step = 1;
                        break;

                    case 1:
                        // Wait for reply
                        ACLMessage reply = receive();

                        if (reply != null) {
                            System.out.println("Agent A received reply: " + reply.getContent());
                            replyReceived = true;
                        } else {
                            block();
                        }
                        break;
                }
            }

            @Override
            public boolean done() {
                return replyReceived;
            }
        });
    }
}
```

4.3.2 AgentB_Ex2.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;

public class AgentB_Ex2 extends Agent {

    @Override
    protected void setup() {
        System.out.println("Agent B (Ex2) is ready and listening...");

        addBehaviour(new CyclicBehaviour() {
            @Override
            public void action() {
                ACLMessage msg = receive();

                if (msg != null) {
                    System.out.println("Agent B received: " + msg.getContent());

                    // Check if it's a REQUEST
                    if (msg.getPerformative() == ACLMessage.REQUEST) {
                        // Create a reply using createReply()
                        ACLMessage reply = msg.createReply();
                        reply.setPerformative(ACLMessage.INFORM);
                        reply.setContent("It is 25 C .");

                        send(reply);
                        System.out.println("Agent B sent reply: It is 25 C .");
                    }
                } else {
                    block();
                }
            }
        });
    }
}
```

4.4 Execution Command

```
-gui -local-port 1100 agentB:eu.uemf.agents.AgentB_Ex2;agentA:eu.uemf.agents.AgentA_Ex2
```

4.5 Console Output

```
Console
New_configuration [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.justj.openjdk.hotsp
févr. 17, 2026 10:57:45 AM jade.core.messaging.MessagingService boot
INFOS: MTP addresses:
http://DESKTOP-RBNDAVF:7778/acc
févr. 17, 2026 10:57:45 AM jade.core.AgentContainerImpl joinPlatform
INFOS: ----------------------------------------
Agent container Main-Container@10.12.33.173 is ready.
--------------------------------------------
Agent A (Ex2) is starting...
Agent B (Ex2) is ready and listening...
Agent A sent REQUEST: What is the temperature?
Agent B received: What is the temperature?
Agent B sent reply: It is 25°C.
Agent A received reply: It is 25°C.
```

Figure 3: Bidirectional communication output

5 Exercise 3: Coordination of 3 Agents

5.1 Objective

A Coordinator Agent sends tasks to two Expert Agents and waits for both replies before concluding.

5.2 Implementation

The Coordinator uses a counter to track received replies. When the counter reaches 2, it displays a completion message and removes the listening behavior.

5.3 Complete Code Implementation

5.3.1 CoordinatorAgent.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.OneShotBehaviour;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;

public class CoordinatorAgent extends Agent {
    private int replyCount = 0;

    @Override
    protected void setup() {
        System.out.println("Coordinator is starting...");

```

```java
        // Step 1: Send requests to both experts
        addBehaviour(new OneShotBehaviour() {
            @Override
            public void action() {
                // Send to Expert 1
                ACLMessage msg1 = new ACLMessage(ACLMessage.REQUEST);
                msg1.addReceiver(new AID("expert1", AID.ISLOCALNAME));
                msg1.setContent("Task for Expert 1");
                send(msg1);
                System.out.println("Coordinator sent task to Expert 1");

                // Send to Expert 2
                ACLMessage msg2 = new ACLMessage(ACLMessage.REQUEST);
                msg2.addReceiver(new AID("expert2", AID.ISLOCALNAME));
                msg2.setContent("Task for Expert 2");
                send(msg2);
                System.out.println("Coordinator sent task to Expert 2");
            }
        });

        // Step 2: Listen for replies
        addBehaviour(new CyclicBehaviour() {
            @Override
            public void action() {
                ACLMessage reply = receive();

                if (reply != null && reply.getPerformative() ==
    ACLMessage.INFORM) {
                    replyCount++;
                    System.out.println("Coordinator received reply " +
    replyCount + ": " + reply.getContent());

                    if (replyCount == 2) {
                        System.out.println("All replies received!");
                        // Remove this behavior
                        myAgent.removeBehaviour(this);
                    }
                } else {
                    block();
                }
            }
        });
    }
}
```

5.3.2 Expert1.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;

public class Expert1 extends Agent {

    @Override
    protected void setup() {
```

```java
        System.out.println("Expert 1 is ready...");

        addBehaviour(new CyclicBehaviour() {
            @Override
            public void action() {
                ACLMessage msg = receive();

                if (msg != null && msg.getPerformative() == ACLMessage.REQUEST) {
                    System.out.println("Expert 1 received: " + msg.getContent());

                    // Create reply
                    ACLMessage reply = msg.createReply();
                    reply.setPerformative(ACLMessage.INFORM);
                    reply.setContent("Expert 1 completed the task");
                    send(reply);

                    System.out.println("Expert 1 sent reply");
                } else {
                    block();
                }
            }
        });
    }
}
```

5.3.3 Expert2.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;

public class Expert2 extends Agent {

    @Override
    protected void setup() {
        System.out.println("Expert 2 is ready...");

        addBehaviour(new CyclicBehaviour() {
            @Override
            public void action() {
                ACLMessage msg = receive();

                if (msg != null && msg.getPerformative() == ACLMessage.REQUEST) {
                    System.out.println("Expert 2 received: " + msg.getContent());

                    // Create reply
                    ACLMessage reply = msg.createReply();
                    reply.setPerformative(ACLMessage.INFORM);
                    reply.setContent("Expert 2 completed the task");
                    send(reply);

```

```
                    System.out.println("Expert 2 sent reply");
                } else {
                    block();
                }
            }
        });
    }
}
```

5.4 Execution Command

```
-gui -local-port 1101 coordinator:eu.uemf.agents.CoordinatorAgent
   ;expert1:eu.uemf.agents.Expert1;expert2:eu.uemf.agents.Expert2
```

5.5 JADE GUI Interface

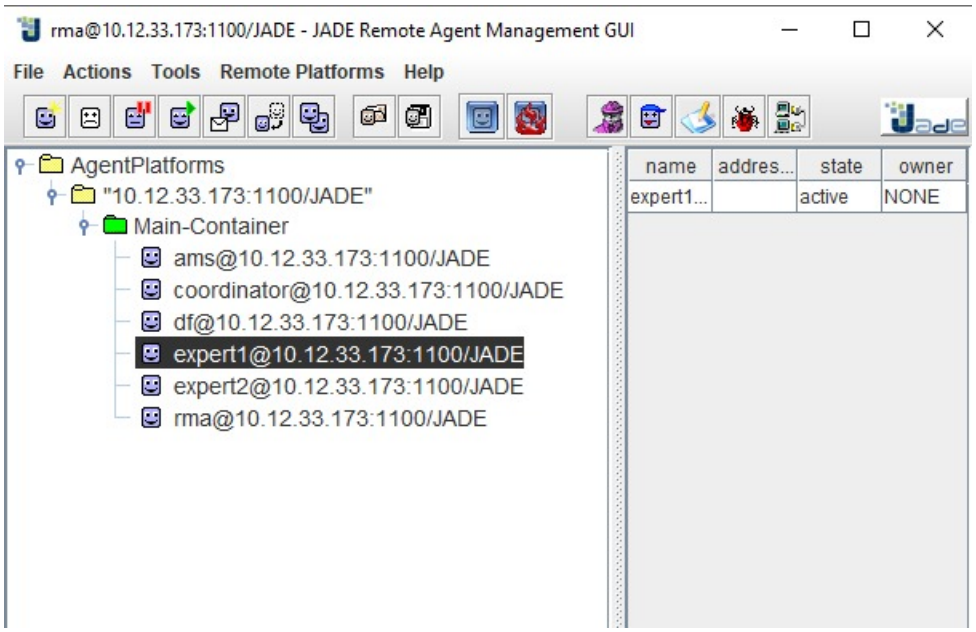

Figure 4: JADE GUI showing Coordinator and two Experts

5.6 Console Output

```
Console
New_configuration [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.justj.openj
févr. 17, 2026 11:10:46 AM jade.core.AgentContainerImpl joinPlatform
INFOS: ----------------------------------------
Agent container Main-Container@10.12.33.173 is ready.
--------------------------------------------
Coordinator is starting...
Expert 2 is ready...
Coordinator sent task to Expert 1
Coordinator sent task to Expert 2
Expert 1 is ready...
Expert 1 received: Task for Expert 1
Expert 2 received: Task for Expert 2
Expert 2 sent reply
Expert 1 sent reply
Coordinator received reply 1: Expert 2 completed the task
Coordinator received reply 2: Expert 1 completed the task
✓ All replies received!
```

Figure 5: Coordination output showing all three agents

6 Exercise 4: Decision Making and Negotiation

6.1 Objective

Implement a negotiation protocol where a Buyer Agent requests proposals from two Seller Agents, compares prices, and accepts the cheapest offer.

6.2 Implementation

The Buyer sends CFP (Call For Proposal) messages to both sellers, collects PROPOSE messages containing prices, compares them, and sends ACCEPT_PROPOSAL to the winner and REJECT_PROPOSAL to the loser.

6.3 Complete Code Implementation

6.3.1 BuyerAgent.java

```java
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.Behaviour;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;

```

```java
public class BuyerAgent extends Agent {

    @Override
    protected void setup() {
        System.out.println("Buyer is starting negotiation...");

        addBehaviour(new Behaviour() {
            private int step = 0;
            private int proposalCount = 0;
            private int minPrice = Integer.MAX_VALUE;
            private AID bestSeller = null;
            private ACLMessage proposal1 = null;
            private ACLMessage proposal2 = null;

            @Override
            public void action() {
                switch (step) {
                    case 0:
                        // Send CFP (Call For Proposal) to both sellers
                        ACLMessage cfp = new ACLMessage(ACLMessage.CFP);
                        cfp.addReceiver(new AID("seller1", AID.ISLOCALNAME));
                        cfp.addReceiver(new AID("seller2", AID.ISLOCALNAME));
                        cfp.setContent("I need parts. What's your price?");
                        send(cfp);

                        System.out.println("Buyer sent CFP to both sellers");
                        step = 1;
                        break;

                    case 1:
                        // Receive proposals
                        MessageTemplate mt = MessageTemplate.MatchPerformative(ACLMessage.PROPOSE);
                        ACLMessage proposal = receive(mt);

                        if (proposal != null) {
                            proposalCount++;
                            int price = Integer.parseInt(proposal.getContent());
                            String sellerName = proposal.getSender().getLocalName();

                            System.out.println("Buyer received proposal from " + sellerName + ": " + price);

                            // Store proposals
                            if (proposal1 == null) {
                                proposal1 = proposal;
                            } else {
                                proposal2 = proposal;
                            }

                            // Track best price
                            if (price < minPrice) {
```

```java
                                minPrice = price;
                                bestSeller = proposal.getSender();
                            }

                            if (proposalCount == 2) {
                                step = 2;
                            }
                        } else {
                            block();
                        }
                        break;

                    case 2:
                        // Send acceptance/rejection
                        System.out.println("\nBuyer compared prices. Best price: " + minPrice);
                        System.out.println("Winner: " + bestSeller.getLocalName() + "\n");

                        // Reply to proposal1
                        ACLMessage reply1 = proposal1.createReply();
                        if (proposal1.getSender().equals(bestSeller)) {
                            reply1.setPerformative(ACLMessage.ACCEPT_PROPOSAL);
                        } else {
                            reply1.setPerformative(ACLMessage.REJECT_PROPOSAL);
                        }
                        send(reply1);

                        // Reply to proposal2
                        ACLMessage reply2 = proposal2.createReply();
                        if (proposal2.getSender().equals(bestSeller)) {
                            reply2.setPerformative(ACLMessage.ACCEPT_PROPOSAL);
                        } else {
                            reply2.setPerformative(ACLMessage.REJECT_PROPOSAL);
                        }
                        send(reply2);

                        step = 3;
                        break;
                }
            }

            @Override
            public boolean done() {
                return step == 3;
            }
        });
    }
}
```

6.3.2 Seller1.java

```java
package eu.uemf.agents;
```

```java

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;

public class Seller1 extends Agent {
    private int myPrice = 500;

    @Override
    protected void setup() {
        System.out.println("Seller 1 is ready (Price: " + myPrice + ")");

        addBehaviour(new CyclicBehaviour() {
            private int step = 0;

            @Override
            public void action() {
                switch (step) {
                    case 0:
                        // Wait for CFP
                        MessageTemplate mtCfp = MessageTemplate.MatchPerformative(ACLMessage.CFP);
                        ACLMessage cfp = receive(mtCfp);

                        if (cfp != null) {
                            System.out.println("Seller 1 received CFP");

                            // Send proposal
                            ACLMessage proposal = cfp.createReply();
                            proposal.setPerformative(ACLMessage.PROPOSE);
                            proposal.setContent(String.valueOf(myPrice));
                            send(proposal);

                            System.out.println("Seller 1 proposed: " + myPrice);
                            step = 1;
                        } else {
                            block();
                        }
                        break;

                    case 1:
                        // Wait for acceptance/rejection
                        ACLMessage response = receive();

                        if (response != null) {
                            if (response.getPerformative() == ACLMessage.ACCEPT_PROPOSAL) {
                                System.out.println("Seller 1: Hooray, I won the deal!");
                            } else if (response.getPerformative() == ACLMessage.REJECT_PROPOSAL) {
                                System.out.println("Seller 1: My proposal was rejected.");
```

```
                            }
                            step = 0; // Reset for next negotiation
                        } else {
                            block();
                        }
                        break;
                    }
                }
            });
        }
}
```

6.3.3 Seller2.java

```
package eu.uemf.agents;

import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;

public class Seller2 extends Agent {
    private int myPrice = 450;

    @Override
    protected void setup() {
        System.out.println("Seller 2 is ready (Price: " + myPrice + ")");

        addBehaviour(new CyclicBehaviour() {
            private int step = 0;

            @Override
            public void action() {
                switch (step) {
                    case 0:
                        // Wait for CFP
                        MessageTemplate mtCfp = MessageTemplate.MatchPerformative(ACLMessage.CFP);
                        ACLMessage cfp = receive(mtCfp);

                        if (cfp != null) {
                            System.out.println("Seller 2 received CFP");

                            // Send proposal
                            ACLMessage proposal = cfp.createReply();
                            proposal.setPerformative(ACLMessage.PROPOSE);
                            proposal.setContent(String.valueOf(myPrice));
                            send(proposal);

                            System.out.println("Seller 2 proposed: " + myPrice);
                            step = 1;
                        } else {
                            block();
```

```
                        }
                        break;

                    case 1:
                        // Wait for acceptance/rejection
                        ACLMessage response = receive();

                        if (response != null) {
                            if (response.getPerformative() == ACLMessage.ACCEPT_PROPOSAL) {
                                System.out.println("Seller 2: Hooray, I won the deal!");
                            } else if (response.getPerformative() == ACLMessage.REJECT_PROPOSAL) {
                                System.out.println("Seller 2: My proposal was rejected.");
                            }
                            step = 0; // Reset for next negotiation
                        } else {
                            block();
                        }
                        break;
                }
            }
        });
    }
}
```

6.4 Execution Command

```
-gui -local-port 1102 buyer:eu.uemf.agents.BuyerAgent;seller1:eu.uemf.agents.Seller1;seller2:eu.uemf.agents.Seller2
```

6.5 JADE GUI Interface

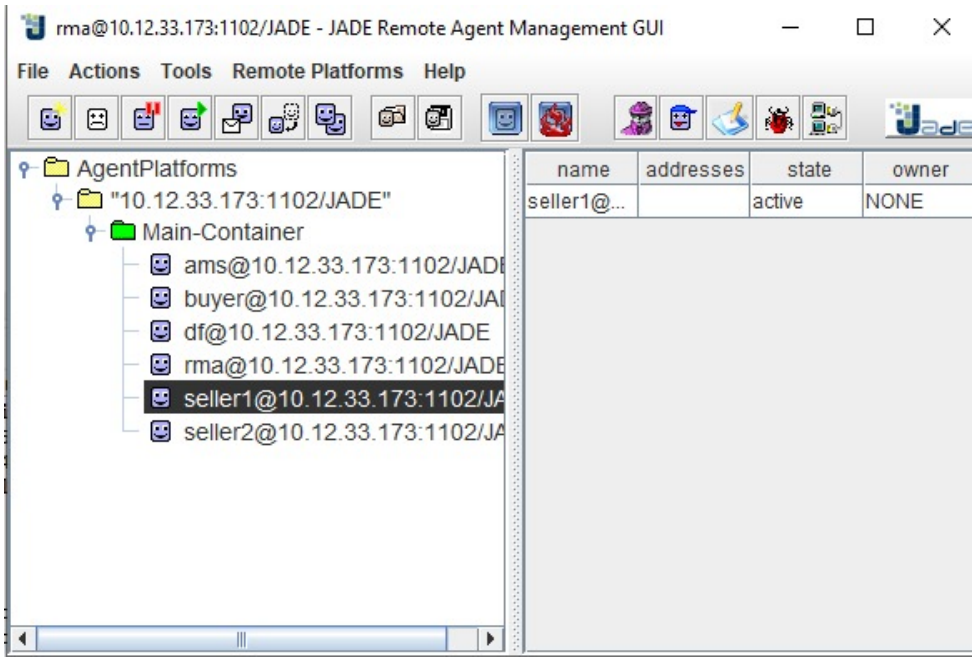

Figure 6: JADE GUI showing Buyer and two Sellers

6.6 Console Output

```
Console
New_configuration [Java Application] C:\Users\HP\.p2\pool\plugins\org
Buyer is starting negotiation...
Seller 1 is ready (Price: 500)
Seller 2 is ready (Price: 450)
Buyer sent CFP to both sellers
Seller 2 received CFP
Seller 1 received CFP
Seller 1 proposed: 500
Seller 2 proposed: 450
Buyer received proposal from seller1: 500
Buyer received proposal from seller2: 450

Buyer compared prices. Best price: 450
Winner: seller2

✗ Seller 1: My proposal was rejected.
✓ Seller 2: Hooray, I won the deal!
```

Figure 7: Negotiation protocol output with winner selection

7 Errors Encountered and Solutions

7.1 Port Already in Use

Error Message:

```
Cannot bind server socket to localhost port 1099
jade.core.IMTPException: No ICP active
```

Cause: A previous JADE instance was still running and occupying port 1099.
Solutions Applied:

1. **Method 1:** Clicked the red stop button in Eclipse console to terminate previous runs
2. **Method 2:** Used Task Manager to kill all `java.exe` processes
3. **Method 3:** Changed port by adding `-local-port [number]` argument to run different exercises simultaneously

8 Conclusion

This lab successfully demonstrated the implementation of ACL communication in JADE through four progressive exercises:

1. **Unidirectional communication:** Basic message sending using INFORM performative
2. **Bidirectional communication:** Request-response pattern using REQUEST and INFORM
3. **Multi-agent coordination:** Synchronization between three agents with reply tracking
4. **Negotiation protocol:** Contract Net Protocol implementation with CFP, PROPOSE, ACCEPT, and REJECT

Key learning outcomes include understanding JADE behaviors (OneShotBehaviour, CyclicBehaviour), ACL message types, agent coordination patterns, and troubleshooting common development issues in Eclipse.